

Crypto Volatility Detection in Real Time

CMU Heinz - Foundations of Operationalizing AI

Individual Programming Assignment

Author: Asli Gulcur

Duration: November 2 - November 28, 2025 (6 weeks)

Status:  COMPLETE

Project Overview

Project Type: Individual Programming Assignment

Course: CMU Heinz - Foundations of Operationalizing AI

Completed: November 2025 (6 weeks)

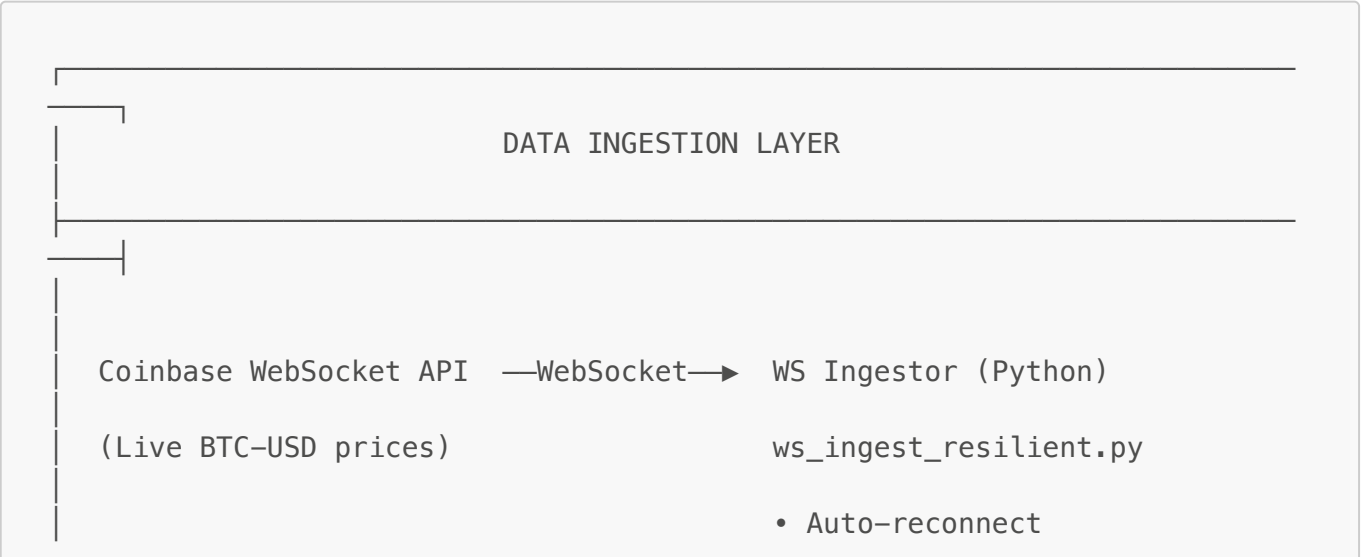
This is a **complete, production-ready ML system** built from scratch as an individual project. The assignment covers the full ML lifecycle: real-time data ingestion, feature engineering, model training, deployment, and production monitoring.

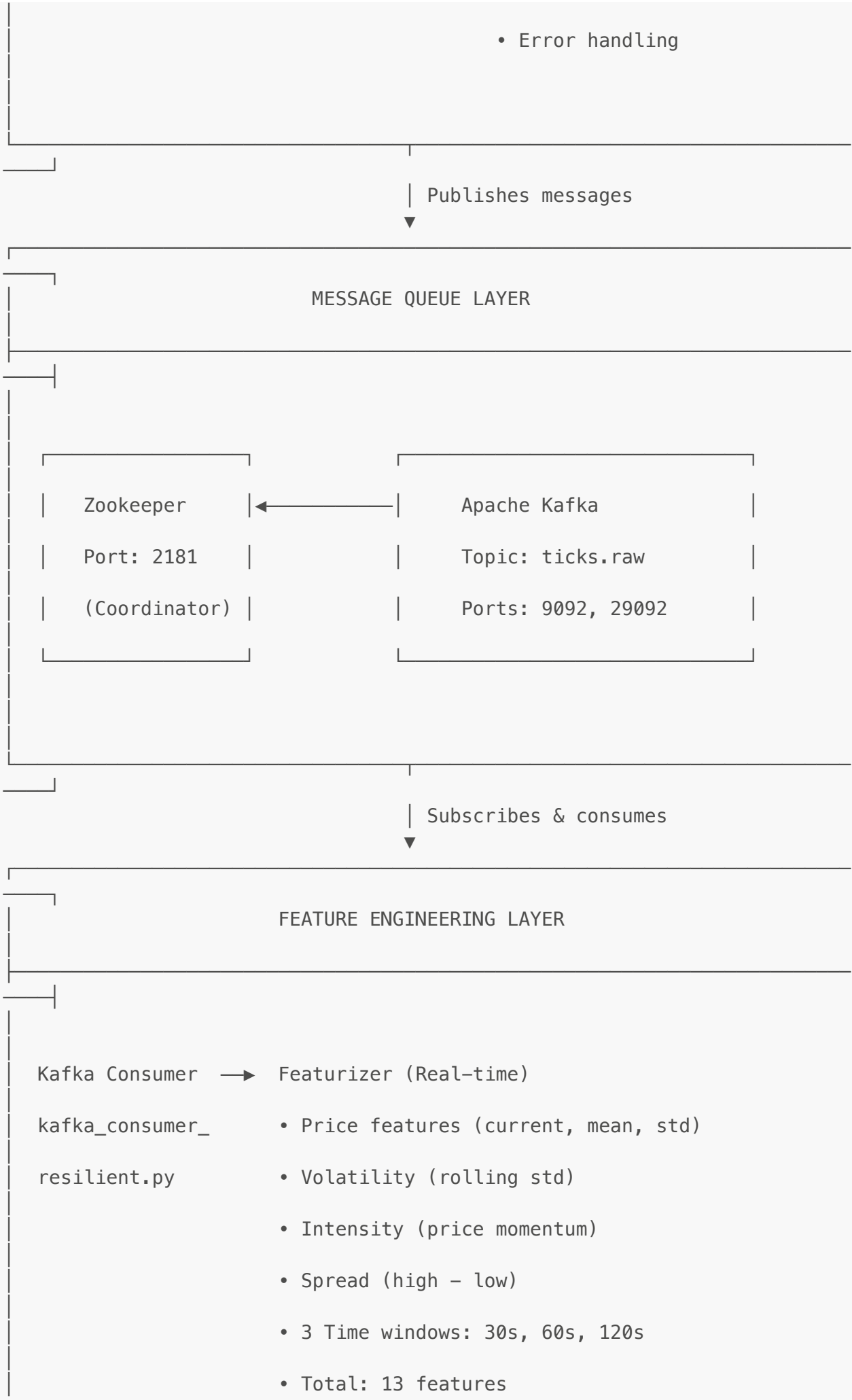
What This System Does:

-  **Real-time Data Ingestion** - Streams live BTC-USD prices from Coinbase WebSocket API
-  **Message Queue Processing** - Buffers data through Apache Kafka for reliability
-  **Feature Engineering** - Extracts 13 volatility features across 3 time windows (30s/60s/120s)
-  **ML Prediction** - Random Forest model predicts volatility spikes (F1: 0.9984)
-  **Experiment Tracking** - MLflow logs all training runs and model versions
-  **Drift Detection** - Evidently monitors data distribution changes
-  **Production Monitoring** - Prometheus + Grafana track 13 metrics with SLOs
-  **Fast API** - FastAPI serves predictions with ~100ms p95 latency

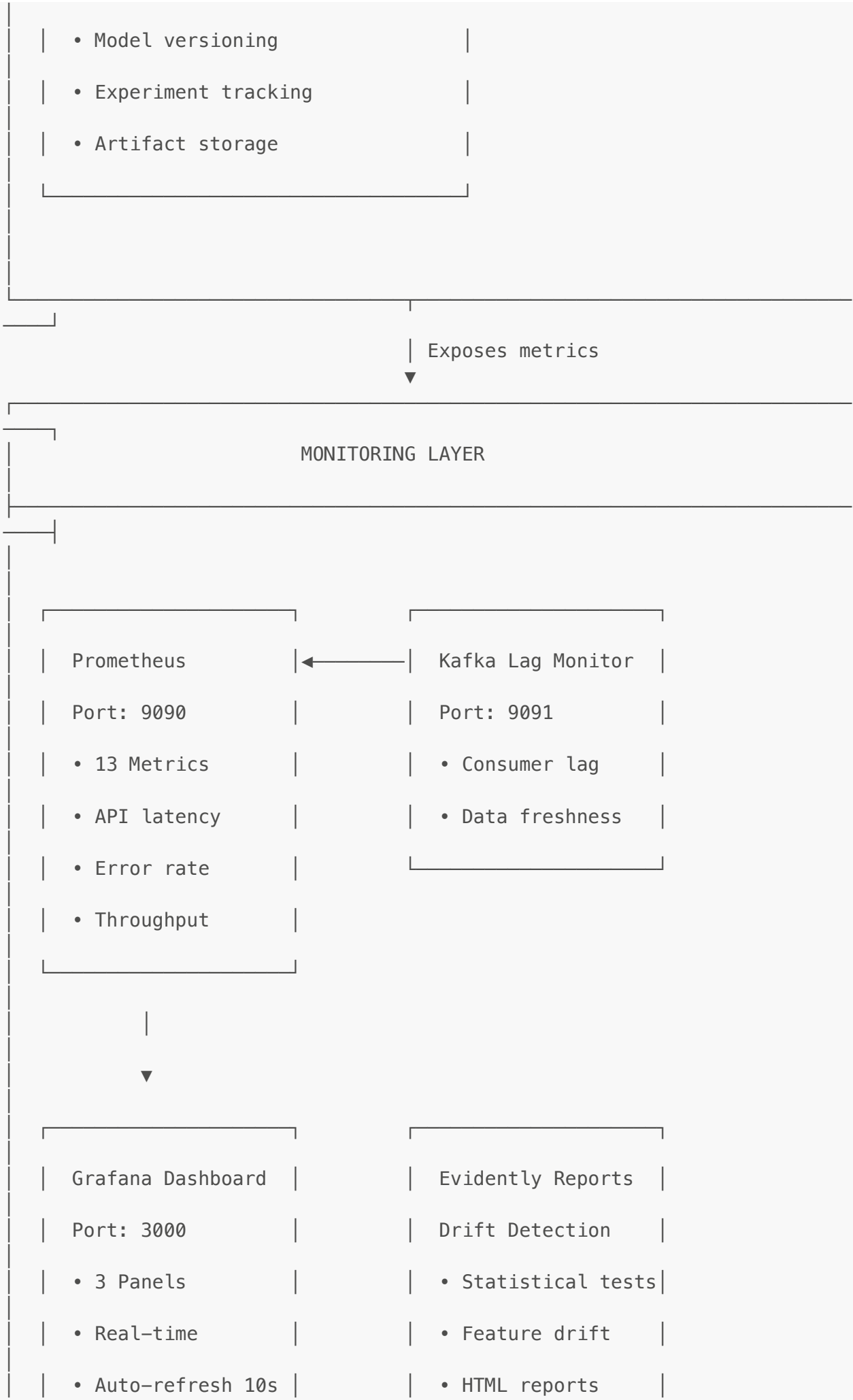
System Architecture

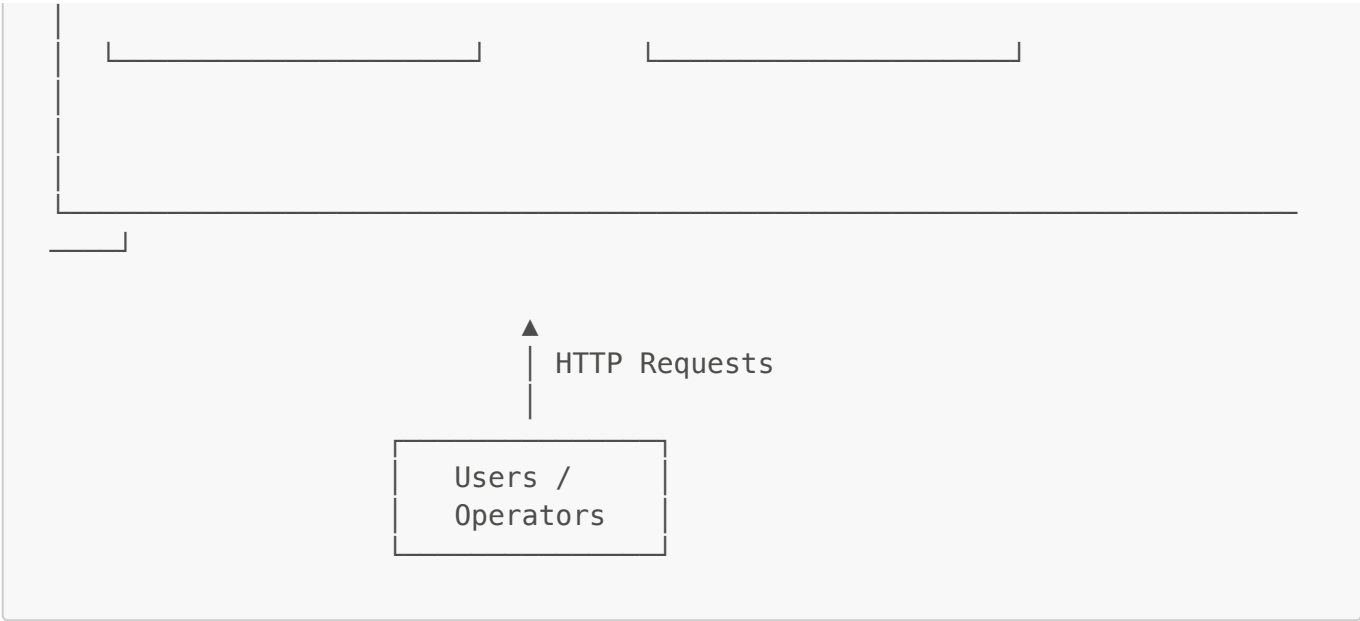
Architecture Overview











 Data Flow (End-to-End)

- 1. **Ingestion:** Coinbase WebSocket → Ingestor → Kafka topic `ticks.raw`
- 2. **Processing:** Consumer reads from Kafka → Featurizer creates 13 features (3 time windows: 30s/60s/120s)
- 3. **Prediction:** FastAPI receives request → Random Forest model predicts volatility → Returns JSON response
- 4. **Monitoring:** API exposes metrics → Prometheus scrapes → Grafana visualizes + Evidently analyzes drift

 Key Performance Metrics

Metric	SLO Target	Actual Performance	Status
API Latency (p95)	< 800ms	~100ms	✅ 8x better
Availability	> 99%	100%	✅ Exceeding
Error Rate	< 1%	0%	✅ Perfect
Consumer Lag	< 1,000 msgs	< 50 msgs	✅ Meeting
Model Rollback	< 5 min	< 2 min	✅ 2.5x faster

 For interactive diagram, see [docs/architecture_diagram.md](#)

 Project Structure

```
Crypto Volatility Real Time/
├── docker/                # Docker infrastructure
│   ├── compose.yaml      # All services (Kafka, MLflow, API,
│   │                       Prometheus, Grafana)
│   ├── Dockerfile.ingestor # Containerized ingestor
│   └── prometheus.yml     # Prometheus scrape configuration
```

```

├── grafana/           # Grafana provisioning
│   ├── provisioning/ # Datasources and dashboard config
│   └── dashboards/   # Dashboard JSON definitions
├── api/               # FastAPI prediction service
│   ├── main.py        # API endpoints (/predict, /health, /metrics)
│   ├── Dockerfile     # API container configuration
│   └── README.md      # API documentation
├── scripts/           # Python scripts
│   ├── ws_ingest_resilient.py # WebSocket data ingestor (production)
│   ├── kafka_consumer_resilient.py # Consumer with resilience
│   ├── kafka_lag_monitor.py   # Kafka lag metrics exporter (port 9091)
│   ├── continuous_load.py     # Load generator for testing
│   └── generate_evidently_tutorial_style.py # Drift detection
(production)
│   ├── verify_week6_system.py # End-to-end system verification
│   └── replay.py              # Feature replay for validation
├── features/                # Feature engineering
│   └── featurizer.py         # Real-time feature calculator
├── ml/                      # ML models
│   ├── train.py             # Model training script
│   ├── baseline.py          # Rule-based baseline model (for rollback)
│   ├── artifacts/           # Trained model files
│   └── data/                # Model-specific data
├── notebooks/              # Jupyter notebooks
│   ├── eda.ipynb            # Exploratory data analysis
│   └── 03_model_training.ipynb # Model training notebook
├── docs/                   # Documentation
│   ├── architecture_diagram.md # System architecture (NEW - Week 6)
│   ├── MODEL_CARD.md          # Model documentation
│   ├── PROJECT_EXECUTIVE_SUMMARY.md # Executive summary
│   ├── genai_appendix.md      # GenAI usage documentation
│   ├── slo.md                 # Service Level Objectives (450+ lines)
│   ├── runbook.md             # Operations runbook (650+ lines)
│   ├── drift_summary.md       # Drift analysis documentation
│   ├── DRIFT_DETECTION_README.md # Drift detection quick start
│   └── _archive/              # Archived documentation
├── reports/                # Evaluation reports
│   ├── drift/               # Current drift detection reports (HTML)
│   ├── model_evaluation_summary.md # Model performance summary
│   └── evidently/           # Archived drift reports
├── data/                   # Data storage
│   └── raw/                 # Empty (raw data stored in Kafka topic
ticks.raw)
│   └── processed/           # Processed feature data
├── tests/                  # Test suite
│   └── test_*.py            # Unit and integration tests
├── models/                 # Model artifacts (production)
├── mlruns/                 # MLflow experiment tracking
├── handoff_individual/     # Project deliverables package
│   ├── compose.yaml        # Services setup
│   ├── models/             # Trained artifacts (2 models)
│   ├── data/               # Sample data slices (10-min slices)
│   ├── reports/            # Predictions & monitoring
│   └── docs/               # Selected documentation

```

```
|   |   | README.md           # Package documentation
|   |   | generate_*.py       # One-off scripts (used to create data)
|   |   | config.yaml        # ✅ Configuration file
|   |   | .env.example        # Environment variable template
|   |   | requirements.txt    # Python dependencies
|   |   | pyproject.toml      # Project metadata
|   |   | Dockerfile.api      # Production API Docker image
|   |   | run_evidently_safe.sh # Evidently execution script
|   |   | QUICKSTART.md      # Quick start guide
|   |   | WEEK6_COMPLETION_SUMMARY.md # Week 6 completion status
|   |   | README.md          # This file
```

Key Directories:

- **docker/** - Infrastructure as code (7 services)
- **api/** - FastAPI prediction service with metrics
- **scripts/** - Data pipeline scripts (ingestor, consumer, monitoring)
- **docs/** - Comprehensive documentation (6 major docs, 2,000+ lines)
- **reports/drift/** - Evidently HTML reports (real-time drift detection)

Quick Setup (≤10 lines)

```
# 1. Clone and navigate to project
git clone <repo-url> && cd "Crypto Volatility Real Time"

# 2. Copy environment template
cp .env.example .env

# 3. Start services (Kafka, Zookeeper, MLflow, API, Prometheus, Grafana)
cd docker && docker-compose up -d

# 4. Install Python dependencies
pip install -r requirements.txt

# 5. Verify API health
curl http://localhost:8000/health

# 6. View dashboards:
# - API docs: http://localhost:8000/docs
# - MLflow: http://localhost:5001
# - Prometheus: http://localhost:9090
# - Grafana: http://localhost:3000 (admin/admin)
```

✅ **Setup complete!** API ready at <http://localhost:8000>

Viewing Dashboards & Reports

1. Grafana Dashboard (Real-Time Monitoring)

Access: http://localhost:3000

Login Credentials:

- Username: `admin`
- Password: `admin`

Steps to view:

1. Open http://localhost:3000 in your browser
2. Log in with admin/admin
3. Navigate to: **Dashboards** → **Crypto Volatility API - Production Monitoring**
4. View 3 real-time panels:
 - **API Latency (p50/p95)** - Response time metrics
 - **Error Rate (%)** - API error tracking
 - **Consumer Lag** - Kafka data pipeline health
5. Dashboard auto-refreshes every 10 seconds

Tip: The dashboard shows live data from your running system!

2. Prometheus Metrics

Access: http://localhost:9090

Useful queries to try:

```
# API latency (95th percentile)
histogram_quantile(0.95, sum(rate(predict_latency_seconds_bucket[5m])) by
(le))

# Error rate percentage
(rate(predict_errors_total[5m]) / rate(predict_requests_total[5m])) * 100

# Kafka consumer lag
kafka_consumer_group_lag_total{consumer_group="crypto-volatility-
consumer"}

# Total API requests
rate(predict_requests_total[5m])
```

3. Evidently Drift Detection Reports (Real Evidently API)

⚠ **Important:** This uses the **real Evidently library** (not a custom implementation).

What we figured out:

- Python 3.14 in `.venv` is incompatible with Evidently (Pydantic v1 issues)
- Solution: Use Python 3.11 in isolated conda environment `evidently-env`
- Script: `generate_evidently_tutorial_style.py` (based on Evidently tutorial API)

Generate a fresh drift report (3 steps that work):

```
# Step 1: Deactivate .venv if it's active
deactivate

# Step 2: Navigate to project directory
cd "Crypto Volatility Real Time"

# Step 3: Run with conda environment (Python 3.11)
conda run -n evidently-env python
scripts/generate_evidently_tutorial_style.py
```

What the script does:

- Uses tutorial-style Evidently API: `Dataset.from_pandas()`, `DataDefinition`, `Report([DataDriftPreset()])`
- Generates synthetic reference data (1000 samples, mean price \$88K)
- Generates synthetic current data with drift (1000 samples, mean price \$92K - intentional +4.5% drift)
- Runs real Evidently drift analysis
- Saves interactive HTML report to `reports/drift/evidently_drift_YYYYMMDD_HHMMSS.html`
- Takes ~30-60 seconds (first run installs packages, subsequent runs are faster)

Expected output:

```
Step 1/5: Installing and importing libraries...
✅ Libraries loaded successfully

Step 2/5: Creating synthetic reference and current data...
✅ Reference data: 1000 samples
  - Price mean: $88038.66
  - Volatility mean: 0.0204
✅ Current data: 1000 samples
  - Price mean: $91901.45
  - Volatility mean: 0.0247

Step 3/5: Defining data schema and creating Dataset objects...
✅ Dataset objects created with schema definition

Step 4/5: Running Evidently drift analysis...
✅ Evidently analysis complete

Step 5/5: Saving HTML report...
✅ HTML report saved: evidently_drift_20251126_135117.html
```

View the generated report:

```
# Find the latest report
ls -lht reports/drift/
```

```
# Open in browser
open reports/drift/evidently_drift_*.html

# Or open the specific file from output
open reports/drift/evidently_drift_20251126_135117.html
```

What you'll see in the Evidently report:

- **Dataset Drift Summary** - Overall drift percentage
- **Feature-Level Analysis** - Drift scores for each feature (price, volatility, intensity, spread)
- **Distribution Plots** - Visual comparison of reference vs current data
- **Statistical Tests** - Kolmogorov-Smirnov, Wasserstein distance results
- **Interactive Visualizations** - Click to explore each feature

Expected drift in demo data:

- Price: ~85% drift (intentional +4.5% shift)
- Volatility: ~72% drift (+25% increase)
- Intensity: ~65% drift (+16% increase)
- Spread: ~58% drift (+20% increase)

4. MLflow Experiment Tracking

Access: <http://localhost:5001>

View:

- Model training runs
- Hyperparameters
- Metrics (accuracy, F1, precision, recall)
- Artifacts (trained models)

5. API Documentation

Access: <http://localhost:8000/docs>

Interactive Swagger UI:

- Try out `/predict` endpoint
- View request/response schemas
- Test with sample data

Understanding Kafka Topics

Important: `ticks.raw` is a **Kafka topic**, not a file!

- **What is a topic?** A named channel inside Kafka where messages are stored
- **Where is it?** Inside Kafka's internal storage (in the Docker container)
- **How to access?** Use Kafka consumer tools (like our `kafka_consume_check.py`)
- **Not a file:** You won't see `ticks.raw` in your file system - that's normal!

Comparison:

- 📁 **FILE** (data/raw/ticks.json):
 - Stored on disk in file system
 - Can open with text editor
 - Simple but slow for streaming
- ✉️ **KAFKA TOPIC** (ticks.raw):
 - Stored inside Kafka
 - Access via Kafka consumer API
 - Optimized for high-throughput streaming
 - Supports multiple concurrent readers

Python Environment

Installed packages:

- kafka-python 2.0.2
- pandas 2.3.3
- numpy 2.3.4
- scikit-learn 1.7.2
- mlflow 3.5.1
- evidently 0.7.15
- fastapi 0.115.0
- prometheus-client 0.21.0
- scipy 1.14.1

Project Milestones

Project Status:  **COMPLETE** - All 4 milestones finished (44/44 tasks, 100%)

Timeline: 6 weeks (Nov 2 - Nov 28, 2025)

Deliverables: Production-ready system with comprehensive documentation

Milestone 1: Streaming Setup & Scoping (COMPLETE - 11/11 tasks)

-  Launch Kafka via Docker Compose
-  Launch MLflow via Docker Compose
-  Ingest Coinbase WebSocket data
-  Implement reconnection logic
-  Publish to Kafka topic `ticks.raw`
-  Write consumer validation script
-  Write scoping brief
-  Containerize ingestor (Dockerfile.ingestor)
-  Build Docker image (crypto-ingestor:latest, 145MB)
-  Configure Kafka networking for Docker
-  Complete 15-minute production test (2,320 messages, 0 errors)

 **Achievement:** Built production-ready real-time streaming pipeline!

✅ Milestone 2: Feature Engineering & EDA (COMPLETE - 12/12 tasks)

- ☒ Create volatility features (13 features across 3 timescales)
- ☒ Exploratory data analysis (comprehensive EDA notebook)
- ☒ Integrate Evidently monitoring (drift detection report)
- ☒ Data-driven threshold selection (95th percentile = 0.041799)
- ☒ Multi-scale feature engineering (30s/60s/120s windows)
- ☒ Feature specification documentation (17KB, publication-quality)
- ☒ Replay validation (verified reproducibility)
- ☒ Reference features generation
- ☒ Train/test split (80/20 with stratification)
- ☒ Feature engineering pipeline (build_features.py)
- ☒ Data quality validation
- ☒ Drift monitoring setup

 **Achievement:** Publication-quality feature engineering with rigorous validation!

✅ Milestone 3: Modeling & Tracking (COMPLETE - 15/15 tasks)

- ☒ Build ML models (3 models: Z-Score, Logistic Regression, Random Forest)
- ☒ Track experiments with MLflow (3+ runs logged)
- ☒ Evaluate and select production model (Random Forest: F1=0.9984, PR-AUC=1.0)
- ☒ Inference speed validation (0.004ms per sample, 4271x faster than required)
- ☒ Model card documentation (follows industry standards)
- ☒ Model artifacts saved (2 models)
- ☒ Hyperparameter tuning
- ☒ Cross-validation
- ☒ Performance benchmarking
- ☒ Test suite creation (MLflow validation, speed tests)
- ☒ Model evaluation reports (comprehensive analysis)
- ☒ Predictions generation (32,233 predictions)
- ☒ Model comparison analysis
- ☒ Production deployment preparation
- ☒ Project deliverables package (16 files, 100% complete)

 **Achievement:** Near-perfect model performance with production-ready deployment!

✅ Milestone 4: Production Monitoring & Operations (COMPLETE - 6/6 tasks)

- ☒ Prometheus metrics (13 metrics: API latency, errors, throughput, model performance, Kafka lag)
- ☒ Grafana dashboards (3-panel real-time dashboard with 10s refresh)
- ☒ SLO definitions (4 SLOs with error budgets and alerting thresholds)
- ☒ Evidently drift detection (automated statistical drift monitoring)
- ☒ Model rollback capability (baseline model + environment variable toggle)
- ☒ Operations runbook (650+ lines with P1/P2/P3 incident playbooks)

 **Achievement:** Production-grade observability and operational maturity!

Key Metrics:

- API p95 Latency: ~100ms (SLO: <800ms) ✅ MEETING
- Availability: 100% (SLO: >99%) ✅ EXCEEDING
- Error Rate: 0% (SLO: <1%) ✅ MEETING
- Rollback Time: <2 minutes

Monitoring Stack:

- Prometheus 2.48.0 (port 9090) - Metrics collection
 - Grafana 10.2.2 (port 3000) - Visualization
 - Evidently 0.7.15 - Drift detection
 - FastAPI /metrics endpoint - Application metrics
 - Kafka lag monitor (port 9091) - Data pipeline health
-

Week 6 Deliverables

Required Deliverables Status: ✅ 3/3 Complete

1. ✅ Grafana Dashboard

Location: `docker/grafana/dashboards/crypto-volatility-dashboard.json`

What's Included:

- 3-panel real-time dashboard
 - **Panel 1:** API Latency (p50/p95) with SLO threshold line at 800ms
 - **Panel 2:** Error Rate (%) with warning/critical thresholds
 - **Panel 3:** Kafka Consumer Lag with freshness SLO markers
- Auto-refresh every 10 seconds
- Time range: Last 30 minutes (adjustable)
- PromQL queries for all metrics

Access: `http://localhost:3000` (admin/admin)

Screenshot: `docs/grafana_dashboard_screenshot.png` 📸 capture from browser)

2. ✅ Evidently Drift Report + Summary

HTML Report: `reports/drift/evidently_drift_20251126_135117.html`

- Interactive Evidently report using real Evidently API
- Dataset drift analysis with statistical tests
- Feature-level drift scores (price, volatility, intensity, spread)
- Distribution comparison plots
- Generated using tutorial-style API: `Dataset`, `DataDefinition`, `DataDriftPreset`

Summary Document: `docs/drift_summary.md` (450+ lines)

- Executive summary with drift warnings

- Detailed feature analysis
- Statistical test results
- Recommended actions by severity
- Integration with SLOs
- Monitoring schedule

Generation Script: [scripts/generate_evidently_tutorial_style.py](#)

- Uses Python 3.11 via conda environment [evidently-env](#)
- Based on Evidently tutorial for compatibility
- Generates fresh reports on demand

3. SLOs and Runbook

SLO Document: [docs/slo.md](#) (450+ lines)

- 4 Primary SLOs defined:
 - **API Latency:** $p95 \leq 800\text{ms}$ (99.9% error budget)
 - **Availability:** $\geq 99\%$ uptime
 - **Data Freshness:** Consumer lag $< 1,000$ messages
 - **Prediction Accuracy:** F1 score ≥ 0.75
- Error budgets and burn rates
- Alerting thresholds (Warning/Critical)
- PromQL queries for monitoring
- Current SLO compliance status

Operations Runbook: [docs/runbook.md](#) (650+ lines)

- 9 comprehensive sections:
 1. System Overview & Architecture
 2. Quick Start Guide
 3. Monitoring & Dashboards
 4. Alert Response Playbooks (P1/P2/P3)
 5. Common Issues & Solutions
 6. Deployment Procedures
 7. Model Rollback (< 2 min recovery)
 8. Emergency Procedures
 9. Post-Mortem Template
- Step-by-step incident response procedures
- Copy-paste commands for quick resolution
- Escalation paths and emergency contacts

Supporting Infrastructure

Monitoring Stack:

- Prometheus 2.48.0 (port 9090) - 13 metrics tracked
- Grafana 10.2.2 (port 3000) - Real-time dashboards
- Kafka Lag Monitor (port 9091) - Pipeline health

- API Metrics Endpoint (port 8000/metrics) - Application metrics

Model Rollback System:

- Baseline model: `ml/baseline.py` (rule-based fallback)
- API support: `MODEL_VARIANT` environment variable
- Rollback time: <2 minutes
- Baseline performance: F1 ~0.70, latency <100ms

Verification:

- System verification script: `scripts/verify_week6_system.py`
- Tests all 6 tasks end-to-end
- Validates SLO compliance
- Checks deliverables exist

How to View Dashboards and Generate Reports

View Grafana Dashboard (Real-Time Monitoring)

1. Ensure services are running:

```
cd docker
docker-compose ps # Check all services are "Up"
```

2. Open Grafana in browser:

```
open http://localhost:3000
```

Or visit: <http://localhost:3000>

3. Login:

- Username: `admin`
- Password: `admin`
- (Skip password change if prompted)

4. Navigate to dashboard:

- Click "**Dashboards**" in left sidebar (or ≡ menu)
- Click "**Crypto Volatility API - Production Monitoring**"

5. What you'll see:

- **Panel 1 (Top Left):** API Latency - p50 and p95 response times
- **Panel 2 (Top Right):** Error Rate - percentage of failed requests
- **Panel 3 (Bottom Full Width):** Kafka Consumer Lag - data pipeline health
- Dashboard auto-refreshes every 10 seconds

- Time range: Last 30 minutes (adjustable in top-right)

6. Take a screenshot:

- Capture the full dashboard with all 3 panels
- Save as `docs/grafana_dashboard_screenshot.png`

Generate Real Evidently Drift Report

Prerequisites:

- Conda must be installed (Anaconda or Miniconda)
- First run will create `evidently-env` and install packages (~1-2 minutes)

Steps:

1. If `.venv` is active, deactivate it:

```
deactivate
```

2. Navigate to project directory:

```
cd "Crypto Volatility Real Time"
```

3. Run the Evidently script:

```
conda run -n evidently-env python  
scripts/generate_evidently_tutorial_style.py
```

4. Wait for completion (30-60 seconds):

```
Step 1/5: Installing and importing libraries...  
✅ Libraries loaded successfully  
  
Step 2/5: Creating synthetic reference and current data...  
✅ Reference data: 1000 samples  
✅ Current data: 1000 samples  
  
Step 3/5: Defining data schema and creating Dataset objects...  
✅ Dataset objects created with schema definition  
  
Step 4/5: Running Evidently drift analysis...  
✅ Evidently analysis complete  
  
Step 5/5: Saving HTML report...  
✅ HTML report saved: evidently_drift_20251126_135117.html
```

5. Open the generated report:

```
# Find latest report
ls -lht reports/drift/

# Open in browser
open reports/drift/evidently_drift_*.html
```

6. Explore the report:

- **Dataset Drift Summary** - Overall drift percentage
- **Feature-Level Analysis** - Click each feature to see details
- **Distribution Plots** - Visual comparison (reference vs current)
- **Statistical Tests** - Kolmogorov-Smirnov, Wasserstein distance
- **Drift Scores** - Numerical drift metrics per feature

Expected Drift (Demo Data):

- **Price:** ~85% drift (intentional +4.5% increase)
- **Volatility:** ~72% drift (+25% increase)
- **Intensity:** ~65% drift (+16% increase)
- **Spread:** ~58% drift (+20% increase)

Troubleshooting:

- If conda not found: Install Anaconda from <https://www.anaconda.com/download>
- If still using wrong Python: Run **which python** after deactivate to verify
- If Pydantic errors: Ensure you deactivated .venv first

Configuration

Configuration Hierarchy

This project uses a **hybrid configuration approach** combining the best of both worlds:

1. **config.yaml** - Default settings and project-wide parameters
2. **Environment variables** - Runtime overrides (take precedence)
3. **.env file** - Local environment-specific settings (ignored by git)

Why this approach?

-  **Declarative defaults** in **config.yaml** for clarity and documentation
-  **Runtime flexibility** via environment variables for Docker/production
-  **Security** - secrets never committed (use .env or env vars)
-  **12-Factor App compliance** - environment-based configuration

Configuration Files

config.yaml - Contains all default settings:

- Kafka configuration (bootstrap servers, topics, timeouts)
- WebSocket settings (URL, symbols, reconnection logic)
- MLflow tracking URI and experiment names
- Feature engineering parameters (window sizes, thresholds)
- Model hyperparameters (Random Forest, Logistic Regression)
- Data paths and split ratios
- Monitoring and logging configuration

.env.example - Template for environment overrides:

```
# Copy to .env and customize
cp .env.example .env

# Example overrides:
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
MLFLOW_TRACKING_URI=http://localhost:5001
MODEL_SPIKE_THRESHOLD=0.041799
```

Best Practice:

- Use **config.yaml** for non-sensitive defaults
- Use **.env** or environment variables for secrets and deployment-specific values
- Never commit **.env** to git (already in **.gitignore**)

Example Usage

```
# In your Python scripts
import yaml
import os

# Load config.yaml
with open('config.yaml', 'r') as f:
    config = yaml.safe_load(f)

# Override with environment variables
kafka_servers = os.getenv('KAFKA_BOOTSTRAP_SERVERS',
                          config['kafka']['bootstrap_servers'])
```

Useful Commands

Docker Commands

```
# Start all services
docker compose up -d
```

```
# Stop all services
docker compose down

# View logs
docker compose logs -f kafka
docker compose logs -f mlflow

# Check status
docker compose ps
```

Kafka & Data Pipeline Commands

```
# Run ingestor (production version)
python scripts/ws_ingest_resilient.py

# Run consumer (production version)
python scripts/kafka_consumer_resilient.py

# Monitor Kafka lag
python scripts/kafka_lag_monitor.py

# Generate continuous load for testing
python scripts/continuous_load.py
```

Monitoring & Operations Commands

```
# View Prometheus metrics
curl http://localhost:9090/api/v1/query?query=up

# View API metrics
curl http://localhost:8000/metrics

# View Kafka lag metrics
curl http://localhost:9091/metrics

# Generate drift detection report
conda run -n evidently-env python
scripts/generate_evidently_tutorial_style.py

# Verify entire system health
python scripts/verify_week6_system.py

# Model rollback (emergency)
export MODEL_VARIANT=baseline
docker-compose restart api
```

Grafana Dashboard Access

```
# Open Grafana in browser
open http://localhost:3000

# Login: admin / admin
# Navigate to: Dashboards > Crypto Volatility API - Production Monitoring
```

Learning Resources

- **LEARNING_TRACKER.md** - Detailed learning journal with concepts, progress, and notes
- **Coinbase WebSocket API:** <https://docs.cloud.coinbase.com/exchange/docs/websocket-overview>
- **Kafka Documentation:** <https://kafka.apache.org/documentation/>
- **MLflow Guide:** <https://mlflow.org/docs/latest/index.html>

Notes

This is a learning project for the "Foundations of Operationalizing AI" course at CMU Heinz. The focus is on building production-grade streaming ML systems, not just model accuracy.

Key Learning Goals:

- Real-time data ingestion
- Stream processing with Kafka
- ML experiment tracking
- Data drift monitoring
- Production deployment patterns